Ders 8 Genişletilmiş Notu: Bağlam Farkında Sinirsel Dil Modelleri

Ders 8 Genişletilmiş Notu: Bağlam Farkında Sinirsel Dil Modelleri

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Bağlam Farkında Modellere Neden İhtiyaç Var?
- 2. Dağıtık Temsil (Distributed Representations)
- 3. Tekrarlayan Sinir Ağları (RNN)
- 4. RNN'lerle Dil Çevirisi
- 5. LSTM ve GRU
- 6. Attention ve Transformer'lar
- 7. Bağlam Tabanlı Dil Modelleri
- 8. Self-Supervised Görevler
- 9. Byte Pair Encoding
- 10. Büyük Resim ve Hazır Modeller
- 11. BERT Nasıl Çalışır?
- 12. BERT'in İnce Ayarı ve Kullanımı
- Örnek Sorular
- Dersin Toparlaması

1. Bağlam Farkında Modellere Neden İhtiyaç Var?

Dersin slayt dizisinin başlığı "Bağlam Farkında Sinirsel Dil Modelleri"dir (ekrandan, 00:10:00). Hoca konuyu önceki derslerde görülen anlamsal uzaylarla bağladı. O uzaylarda her kelimeye bir vektör atanıyor, kelimeler arasındaki anlamsal benzerlik de string benzerliğinin ötesinde, bu vektörlerin benzerliğiyle ölçülüyordu.

Bu yaklaşımın bir sıkıntısı vardır: bir kelime birden fazla anlama gelse bile ona tek bir vektör atanır. Hoca "çay" örneğini verdi. İçilen çay ile küçük dere anlamındaki çayın vektörlerinin farklı olması gerekir. Hangi çaydan söz edildiği etrafındaki kelimelerden anlaşılır: "çay içiyorum" ile "akan çayın yanına indim" cümlelerinde bağlam anlamı belirler. Bağlam farkında sinirsel dil modelleri, kelimenin vektörünün etrafındaki kelimelere göre değişebilmesini sağlar.

Hoca dersin planını şöyle sıraladı: önce dağıtık temsil, sonra RNN'ler ve türevleri, self-supervised learning, byte pair encoding, bağlam tabanlı dil modellerinden örnekler ve en sonda BERT'in özellikle kullanım açısından daha ayrıntılı anlatımı.

2. Dağıtık Temsil (Distributed Representations)

Hoca bu bölümün amacını derin öğrenmede çok katmanlı yapıların neden işe yaradığına dair kavramsal bir fikir vermek olarak tanımladı; matematiksel bir kanıt değil.

2.1 Sekiz sınıf mı, üç alt özellik mi?

Slayttaki örnek (ekrandan, 00:07:40): resimlerdeki insanları kadın/erkek, gözlüklü/gözlüksüz, uzun/kısa diye belirlemek istiyoruz. Bunun iki yolu vardır.

- Sekiz sınıf tanımlamak: "kadın, gözlüklü, uzun" birinci sınıf, "kadın, gözlüklü, kısa" ikinci sınıf, "erkek, gözlüklü, uzun" üçüncü sınıf gibi. Üç özelliğin her biri iki değerli olduğu için $2^3 = 8$ sınıf olur. Bu durumda her sınıftan örneğe ihtiyaç vardır.
- Üç alt özelliği öğrenen üç ikili (binary) sınıflandırıcı tanımlamak: biri kadın/erkek, biri gözlüklü/gözlüksüz, biri uzun/kısa ayrımını öğrenir. Eğitim kümesinde "uzun, kadın, gözlüksüz" bir örnek olmasa bile, her alt özellik ayrı ayrı görüldüğü için bu kombinasyon tanınabilir.

Genel kural slaytta şöyle verildi: d adet alt özellikle 2^d adet farklı kombinasyon ifade edilebilir. N adet sınıf böyle örtüşmeli (non mutually exclusive) özellikler taşıyorsa $\log_2 N$ adet alt özellikle ifade edilebilir.

2.2 Parametre sayısı örneği

İkinci slayt (ekrandan, 00:10:00) aynı fikri parametre sayısı üzerinden gösterir. 1024 sınıf olsun ve $d \times d$'lik resimler sınıflandırılsın.

- Klasik yaklaşım: giriş katmanı $d \cdot d$ uzunluğunda bir vektördür (resim düzleştirilir), çıkış katmanı 1024 boyutludur. Girişten çıkışa eşleme yapan tek bir W öğrenilir ve bu W'nin $(d \cdot d) \cdot 1024$ parametresi vardır. Slaytta bu "1024 prototip öğrenmek" diye ifade edilir.
- Dağıtık yaklaşım: 1024 sınıfın 10 farklı alt özelliğin kombinasyonlarıyla ifade edilebildiği düşünülür. Girişle çıkış arasına 10 nöronlu bir katman eklenir. Artık iki matris vardır: girişten orta katmana W_1 ($d \cdot d \times 10$) ve orta katmandan çıkışa W_2 (10×1024). Son katman alt özelliklerin kombinasyonlarını sınıf etiketlerine eşlemeyi öğrenir. Öğrenilecek parametre sayısı $(d \cdot d) \cdot 10 + 10 \cdot 1024$ olur.

$d = 128$ için adım adım hesap:

- Verilen: $d = 128$, sınıf sayısı 1024, alt özellik sayısı 10.
- Klasik: $128 \cdot 128 = 16\,384$; $16\,384 \cdot 1024 = 16\,777\,216 = 2^{24}$.
- Dağıtık: $16\,384 \cdot 10 = 163\,840$; $10 \cdot 1024 = 10\,240$; toplam $174\,080$.
- Sonuç: slayttaki ifadeyle "katman sayısını artırarak öğrenmemiz gereken parametre sayısını azalttık".

  Ders sonrası not: Slaytta ve hocanın anlatımında klasik yaklaşım için "yaklaşık 4 milyon (2^{24})" deniyor. $2^{24} = 16\,777\,216$, yani yaklaşık 16,8 milyondur (python ile hesaplandı); üs doğru, "4 milyon" yanlıştır. Dağıtık yaklaşım için slayttaki "yaklaşık 180 bin" değeri, hesaplanan 174 080 ile uyumludur.

Hoca bunun "dağıtık öğrenme" fikri olduğunu söyledi: gözlüklü mü, erkek mi, uzun mu, şişman mı gibi alt özellikler öğrenilirse 1024 sınıf bu özelliklerin çeşitli kombinasyonlarıyla ifade edilebilir.

2.3 Ortak parçası olmayan sınıflar

Soru: 1024 sınıf birbirinden tamamen farklıysa, örneğin 1024 ilçenin resimlerini sınıflandırıyorsak ve ortak bir şeyleri yoksa bu yine yapılabilir mi? **Cevap:** Hoca önce renk örneği verdi: 1024 farklı renk sınıfı varsa orta katmandaki alt özellikler belki kırmızı, yeşil, mavi gibi birkaç temel renktir; bunların çeşitli oranlardaki birleşimleri istenen kadar rengi ifade eder. Olay kelimeye ya da renge özgü değildir. Dağıtık temsilin işlemesi için temel bileşenlerin var olması gerekir. Sınıflar arası ortaklıktan değil, bütünlerin parçalardan oluşmasından söz ediliyor. Bütünler parçalardan oluşmuyor, blok blok veriliyorsa yapılacak bir şey yoktur; sistem bir arama tablosuna (look up table) dönüşür.

> **Hocanın vurgusu:** Amaç aynı sınıftaki örnekler arasındaki benzer alt özellikleri ve farklı sınıflar arasındaki farklı alt özellikleri görebilmektir. Alt özellik diye bir şey yoksa öğrenecek bir şey de yoktur.

2.4 Alt özellikleri kim bulur?

Slayt (ekrandan, 00:20:00) şu soruyu sorar: bir sınıfı ifade eden, sınıflar arası ortak olabilecek parçaları (alt özellikleri) önceden nasıl bilebiliriz? Cevap: öğrenelim. Bir MLP'nin her gizli katmanındaki her nöron bir alt özellik öğrenir. Ağırlıkları görselleştiren araçlarla bu incelenebilir. Bir CNN'deki her filtre de bir alt özellik öğrenir.

Slayttaki varsayım: tüm resimler ortak küçük parçalardan oluşur; en alt seviyede basit özellikler, sonrasında bunların kombinasyonları öğrenilir. Şekilde yüzler, arabalar, filler ve sandalyeler için öğrenilmiş filtreler üç seviyede gösterilir. En alt seviyedeki filtreler dört sınıfta da neredeyse aynıdır: farklı açılarda çizgiler, tamamen koyu ya da açık alanlar. Bunlar bir şekilde birleşince yüzün parçaları, başka bir şekilde birleşince arabanın ya da sandalyenin parçaları oluşur; üst katmanlarda da asıl tanınacak nesneler öğrenilir.

2.5 CNN'ler ve kapsül ağları

CNN'lerde belli alt özelliklerin resmin içinde var olup olmadığına bakılır; pooling katmanı bunların nerede olduğunu önemsiz kılar. Bu yüzden bir yüzün parçaları farklı yerlere konsa CNN hâlâ yüz olduğunu düşünebilir. Oysa ağızla gözün yeri değiştirilmiş bir resme yüz demek ister miyiz? Hocanın aktardığına göre Hinton'ın önerdiği kapsül ağları (capsule networks) parçaların birbirine göre konumlarını da işe katar: iki göz bulunduysa bunların yan yana olması gerekir gibi. Bu ilişkiler de örneklerden öğrenilir.

3. Tekrarlayan Sinir Ağları (RNN)

3.1 Neden geçmişe bakmak gerekir?

RNN'ler (tekrarlayan ya da özyinelemeli yapay sinir ağları) uzun zamandır vardır ve girişin bir zaman serisi olduğu durumlar için kullanılır. Hoca tahtada bir döviz kuru örneği çizdi: pazartesiden cumaya kurun değerleri var ve amaç bir sonraki günün değerini tahmin etmek. Yalnız dünkü değere bakmak yetmez. Aynı dünkü değere, geçmişteki iki farklı gidişatla (biri kırmızıyla, biri turuncuyla çizilen) ulaşılmış olabilir ve her birinde yarına dair tahmin farklı olur. Yani tek bir değerle hareket edilemez, geçmişe bakmak gerekir.

Slayt (ekrandan, 00:30:00) bu yapıları girişin sekans olduğu uygulamalar için tanımlar: bir cümlenin kelimeleri, bir filmin kareleri, bir konuşmanın fonemleri. Gerçek dünyadaki birçok şeyin doğal bir sırası vardır ve sıra karıştırılınca anlam kaybolur.

3.2 Temel gösterim

- $h(t)$: t anındaki çıkış, $x(t)$: t anındaki giriş.
- $h(t)$, $x(t)$'ye ve $h(t-1)$'e bağlıdır; aslında $x_0, \dots, x_t$'nin hepsine bağlıdır.

Hoca bunu şöyle açıkladı: çıkış yalnız şu anki girişe değil, daha önceki şeylere de bağlıdır. Ama geçmişteki her şeyi ayrı ayrı tutmak yerine hepsi bir bağlam vektörünün içine sıkıştırılır. h_{t-1}, başlangıçtan o ana kadarki bütün bilgiyi tutar ve bir sonraki değeri tahmin etmeyi sağlar. Geçmiş değerler aşağı doğru gidiyorsa bir tahmin, yukarı doğru gidiyorsa başka bir tahmin üretilir, çünkü h_{t-1} farklıdır.

Slayttaki şekilde solda kendi üzerine dönen okla bir A kutusu, sağda bunun zamanda açılmış hâli vardır: $x_0 \to h_0$, $x_1 \to h_1$, ... ve her A kutusundan bir sonrakine bir ok gider. h_1 yalnız x_1'e değil, önceki zamandan gelen bağlam vektörüne de bağlıdır.

3.3 Klasik ağlar ve CNN'ler de zamanı kullanabilir

Slayta göre bu gösterim klasik ağların da sabit büyüklükte geçmişi işe katabileceğini gösterir: giriş x yerine birleştirilmiş $(x_t, x_{t-1}, \dots, x_{t-k})$ verilir. Hoca örnek verdi: "son 5 günün verisini tek seferde ver ve çıkışı üret" denebilir.

CNN'ler de zamansal bilgi öğrenebilir. Bir cümlenin her kelimesinin vektörü (ör. 300 boyutlu) bir sütun olarak yan yana dizilirse cümle bir matrise, yani bir resme dönüşür; farklı uzunluklar için doldurma (padding) yapılır. Bu matrisin üzerinde 3×300'lük bir filtre kaydırılırsa filtre her konumda kelimenin kendisine ve önceki kelimelere birlikte bakar. Filtre ne kadar genişse

o kadar geniş bir zaman dilimi işe katılır. Slayta göre son zamanlarda RNN'lerin işini yapan CNN'ler önerilmektedir.

Soru: Kelime vektörleri yan yana dizilince bir vektörün üçüncü elemanı komşu vektörün üçüncü elemanının yanına gelir. Piksel gibi düşünülürse dikeyde ve yatayda ilişkili gibi durur, ama gerçekte öyle bir ilişki yok. Bu yöntem başarılı mı? **Cevap:** Normal CNN'de 2×2, 3×3 gibi küçük çekirdekler gezdirilir ve burada öyle yapılsaydı eleştiri haklı olurdu. Ama burada çekirdek kelime boyutunun tamamını kapsar (300×3) ve yalnız kelime ekseninde kaydırılır. Bir kelimenin farklı boyutları kendi sütununda durur, satırlar da bütün kelimelerin aynı boyutlarını gösterir. Var olmayan bir ilişki eklenmez; yan yanalık zaten var olan bir ilişkidir.

3.4 RNN uygulama türleri

Slayttaki şema (ekrandan, 00:40:00) dört yapı gösterir:

- one to one: klasik yapay sinir ağı, bir girişe bir çıkış.
- one to many: tek bir anda bir giriş gelir, bir sekans çıkar. Örnek: resim girer, onu anlatan cümle çıkar.
- many to one: bir sekans girer, tek çıkış üretilir. Örnek: tweet girer, duygu (sentiment) çıkar.
- many to many: sekans girer, sekans çıkar. Örnek: cümle girer, başka bir dildeki karşılığı çıkar. Giriş ve çıkış uzunlukları eşit olmak zorunda değildir; 10 kelimelik bir cümlenin başka dildeki karşılığı 5 kelimelik olabilir.

3.5 RNN'in matematiği

RNN'in bir iç durumu (state) vardır: h_t. Bu durumda o ana kadar gelen girişler birikir. Güncelleme şöyledir:

$$h_t = f(W^{hh} h_{t-1} + W^{hx} x_t)$$

x_t, W^{hx} matrisiyle; önceki durum h_{t-1}, W^{hh} matrisiyle çarpılır; ikisi toplanıp doğrusal olmayan bir fonksiyondan geçirilir ve yeni h_t elde edilir. Hoca matrislerin anlamını şöyle açıkladı:

- W^{hx}: "şöyle bir giriş gelirse şu duruma git"; girişin durumu ne kadar etkilediğini ifade eder.
- W^{hh}: "önceki durum buysa bir sonraki durum ne olur"; Markov yapılarındaki durum geçiş matrisine benzer.

Yani bir tahmin girişe göre, bir tahmin geçmişteki duruma göre üretilir; bu ikisi ağırlıklı biçimde birleştirilip doğrusal olmayan fonksiyondan geçirilerek yeni durum bulunur. Bütün RNN'lerin temel yapısı budur.

3.6 Bir sonraki harfi tahmin örneği

Slayt (ekrandan, 00:44:40): giriş ve çıkış 4 boyutludur (one-hot encoding, sözlük 4 karakter), 3 gizli nöron vardır; RNN'in üzerine bir katman daha eklenmiş ve W_{hy} de öğrenilmektedir. Sözlük h, e, l, o karakterlerinden oluşur. One-hot kodlamada h = (1, 0, 0, 0), e = (0, 1, 0, 0), l = (0, 0, 1, 0) gibi her karakterin yalnız kendi konumu 1'dir.

Girişler klasik ağdaki gibi tek seferde değil, teker teker verilir: önce "h", sonra "e", sonra "l", sonra "l". Her adımda gizli katman güncellenir ve hedef karakterler sırasıyla "e", "l", "l", "o"dur. Slayttaki ilk adımda gizli katman (0.3, -0.1, 0.9), çıkış katmanı (1.0, 2.2, -3.0, 4.1) değerlerini gösterir.

Önceki mimariden fark, durumun çıkışa verilirken bir matrisle daha çarpılmasıdır: W_{hy}, gizli duruma göre çıkışın ne olacağını öğrenir. Böylece öğrenilecek her şey üç matristir: W_{xh}, W_{hh}, W_{hy}. Bunlar düzgün öğrenildiğinde sistemin modeli ortaya konmuş olur.

> **Hocanın vurgusu:** Temel mekanizma bu kadar; bütün RNN dediğimiz şey bu. Bunun LSTM, GRU gibi türevleri vardır.

Slaytta “Bir sonraki harf yerine bir sonraki kelimeyi tahmin etmek için one-hot kullansak? Alternatif?” sorusu da yer alır; derste cevaplanmadı (bkz. Örnek Sorular).

4. RNN’lerle Dil Çevirisi

4.1 Encoder-decoder yapısı

RNN’lerle dil çevirisi (diziden diziye dönüşüm) bir encoder-decoder yapısına dönüşür. Slayttaki denklemler (ekrandan, 00:50:00):

- Encoder: $h_t = \varphi(h_{t-1}, x_t) = f(W^{hh}h_{t-1} + W^{hx}x_t)$
- Decoder: $h_t = \varphi(h_{t-1}) = f(W^{hh}h_{t-1})$
- Çıkış: $y_t = \text{softmax}(W^S h_t)$

Encoder, kaynak dildeki cümlenin kelimelerini teker teker alır. Başlangıçta durum boştur; her yeni kelimede durum hem önceki duruma hem yeni kelimeye göre güncellenir. En sonda elde edilen vektörde bütün cümlenin temsili vardır; yani cümle tek bir vektöre sıkıştırılır.

Decoder’da artık giriş yoktur. Kaynak dilin kelimeleri bitmiştir, her şey tek bir vektörde toplanmıştır ve amaç bu vektörden hedef dilin kelimelerini birer birer üretmektir. Bir kelime durumdan W^S ile üretilir. Sonraki kelimenin farklı olabilmesi için yeni bir duruma geçmek gerekir; bu yüzden durum W^{hh} ile yeniden güncellenir. Slayttaki örnekte Almanca “Echt dicke Kiste” girişinden İngilizce “Awesome sauce” üretilir.

Hoca bu ilk mimarinin belli bir başarıya ulaştığını ama kural tabanlı çevirinin çok ötesinde işler yapamadığını söyledi. Bariz sıkıntısı, bütün cümlenin tek bir vektöre sıkıştırılması ve yeni cümlenin tamamen bu vektörden üretilmesidir; bu kolay öğrenilebilecek bir yapı değildir.

Soru: İlk çıkış kelimesi (“Awesome”) son durum vektöründen üretiliyor; ikinci kelime (“sauce”) nasıl bulunuyor? Bir önceki adımda üretilen kelime mi veriliyor? **Cevap:** Bu yapıda üretilen kelime bir sonraki adıma verilmez. “Awesome”, o anki durumdan W^S ile üretilir. Sonra bu durum W^{hh} ile çarpılarak güncellenir; yeni durumdan “sauce” üretilir. Yani bir sonraki adıma aktarılan şey kelimenin kendisi değil, onu üreten durumdur.

4.2 İyileştirmeler

Slayt (ekrandan, 01:20:00) bazı iyileştirmeleri listeler.

- Tüm cümle tek bir duruma (C) indirgenmiştir; sorun budur.
- Encoder’da cümleyi tersten vermek: kaynakta A, B, C kelimelerinin karşılığı hedefte D, E, F olsun (A → D, B → E, C → F). Normal sırada A ile D arasında uzun bir mesafe vardır. Kaynak cümle C B A diye verilirse A en son girer ve D hemen ardından üretilir; encode ve decode edilecek bilgiler arasındaki mesafe kısalır. Hoca bunun çok büyük bir iyileşme olmadığını söyledi.
- Decoder’daki iyileştirmeler daha etkilidir: $h_t = \varphi(h_{t-1}, C, y_{t-1})$. Durum yalnız h_{t-1}’den değil, bir önceki adımda üretilen kelimeden (y_{t-1}) ve bütün cümlenin sıkıştırıldığı bağlam vektöründen (C) de üretilir. C her adımda hesaba katılır, unutulmaz. Her biri için ayrı bir W vardır.
- Çıkış: $y_t = \text{softmax}\left(W^S \begin{bmatrix} h_t \\ C \end{bmatrix}\right)$; çıkış da yalnız h_t’ye değil C’ye de bağlıdır.

Hoca bu iyileştirmeler sayesinde sistemin bir miktar daha başarılı hâle geldiğini söyledi.

5. LSTM ve GRU

5.1 LSTM’in çıkış noktası

RNN’lerde yeni bir kelime (x_t) geldiğinde bu kelime mutlaka h_t’yi etkiler. Oysa cümlenin anlamını çok değiştirmeyen kelimeler de vardır ve bunların da durumu değiştirmesi istenmeyebilir. Bir yaklaşım bazı girişlerin durumu etkilemesine izin verip bazılarına vermemektir.

İkinci ihtiyaç bağlamın sıfırlanabilmesidir. Makine çevirisinde iki cümlenin arka arkaya verildiğini ve arka arkaya çevrilmesinin beklendiğini düşünelim. Birinci cümle üretildikten sonra

içeride ona dair bir şey kalmamalı; o noktada bir çeşit sıfırlama gerekir. LSTM'ler bu iki temel amaçla, girişin durumu etkileyip etkilemeyeceğini kontrol etmek ve sıfırlayabilmek için geliştirilmiştir. Hoca LSTM'in 1990'larda geliştirilmiş, yeni olmayan bir yöntem olduğunu belirtti.

5.2 LSTM'in yapısı

Klasik RNN'de x_t ile h_{t-1} birleşip bir tanh'tan geçer ve h_t üretilir. LSTM'de içeride iki bağlam vektörü ilerler: altta klasik h_t, üstte hücre durumu C_t. Slayttaki denklemler (ekrandan, 01:15:30):

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i), \quad \tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o), \quad h_t = o_t * \tanh(C_t)$$

Slayttaki açıklamalar: C_{t-1} belli oranda unutulur ya da saklanır; bunu ilk çarpma (X) operatörü yapar ve f_t, h_{t-1} ile x_t'ye göre hesaplanır. "C_t'ye ne eklenecek?" sorusunun cevabı ikinci terimdir: önceki hidden state ve şimdiki girişten bir i_t ve aday değer $\tilde{C}_t$ üretilir, çarpımları eklenir. Böylece C_t'ye bir şey katılıp katılmayacağı ve eskisinin durup durmayacağı denetlenir.

LSTM eğitimi bu dört matrisin (W_f, W_i, W_C, W_o) öğrenilmesidir; RNN'deki W^{hh} ve W^{hx} gibi içerideki dönüşüm matrisleridir.

5.3 GRU

LSTM kadar karmaşık olmayan ama benzer performans gösteren GRU'larda üç matris öğrenilir. Slayt (ekrandan, 01:17:10):

$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t]), \quad r_t = \sigma(W_r \cdot [h_{t-1}, x_t])$$

$$\tilde{h}_t = \tanh(W \cdot [r_t * h_{t-1}, x_t]), \quad h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t$$

Slaytta "LSTM'e göre daha basit. 3 W matris (W_z, W_r, W) öğrenilir." yazar. Hoca GRU'nun neden LSTM'e denk olduğuna dair çok fazla teorik açıklama olmadığını söyledi: aynı görevlerde denenmiş ve benzer başarı gösterdikleri görülmüştür. İkisi de klasik RNN'den daha iyidir.

6. Attention ve Transformer'lar

6.1 Attention (dikkat)

Hoca daha kritik bir gelişme olarak attention'ı tanıttı. Attention başlangıçta tekrarlayan yapılar (RNN, LSTM) için oluşturulmuş bir mekanizmadır; Transformer'larla başlamamıştır. Makine çevirisinde bir kaynak dil ve bir hedef dil vardır. Attention, hedef dilde bir kelime üretilirken kaynak dildeki hangi kelimelerin etkili olduğunu bulmak için tasarlanmıştır.

Slayt (ekrandan, 01:21:30): "Encoder'ın tüm state'leri etkin, ama ne ölçüde etkin (α) olacakları öğreniliyor." Yani her an encoder'ın bütün durumları kullanılır. Bir çıkış kelimesi ("Awesome" gibi) üretilirken encoder'daki $h_1, h_2, h_3, \ldots$ vektörlerinin hepsi çeşitli ağırlıklarla çarpılarak kullanılır.

Soru: CNN'deki gibi hepsini birlikte mi veriyoruz, tek tek vermek yerine? **Cevap:** Önce vektörler üretilir. Şekilde çift yönlü bir LSTM vardır: hesaplar bir ileri bir geri yönde yapılıp birleştirilir. x_1, x_2, x_3 için üretilen vektörler bu LSTM'in çıkışlarıdır. Sonra bu vektörlerin hepsi tek seferde, attention katsayılarıyla ağırlıklandırılarak kullanılır.

Slayttaki formüller:

- $\alpha^{i,j}$: i. çıkışa j. girişin ne kadar etkisi olacağı.
- $\sum_t \alpha^{1,t} = 1$
- $h^t = (\vec{h}^t, \overleftarrow{h}^t)$
- $C_1 = \sum_t \alpha^{1,t} h^t$

Eski yapıda C yalnız en sondaki durumdu; burada her çıkış için ayrı bir C, bütün durumların belli katsayılarla ağırlıklı toplamı olarak üretilir ve bu katsayılar eğitimde öğrenilir.

Ek parametre sayısı: 20 kelimelik bir giriş ve 20 kelimelik bir çıkış varsa $20 \cdot 20 = 400$ katsayı öğrenilir. Genel olarak giriş token sayısı çarpı çıkış token sayısı kadar ek parametre gerekir.

6.2 Transformer'lar

Bir sonraki adımda RNN yapısı tamamen çıkarıldı ve tamamen attention üzerine bir mekanizma kuruldu. Bu "Attention is all you need" adlı makalede duyuruldu: tekrarlayan yapılara gerek yoktur, yalnız attention ile makine çevirisinde çok iyi performans elde edilebilir. Bugünkü en iyi (state of the art) yaklaşımlar temelde Transformer'ları kullanır.

Hoca Transformer'ın ayrıntısına girmedi; ders sayfasındaki bir belgede bunu formal olmayan bir dille anlattığını ve sondaki kaynakların özellikle iyi olduğunu söyledi.

Önceki attention'da "i. çıkışa j. girişin ne kadar etkisi olacağı", yani çıkış üretilirken girişte nerelere bakılacağı öğreniliyordu. Transformer'da buna ek olarak self-attention vardır.

6.3 Self-attention

Self-attention kabaca, bir cümlenin içindeki kelimelerin aynı cümlenin içindeki diğer kelimelere ne kadar bağlı olduğudur. Hoca bir örnek cümle verdi: "Ali okula gelince onu gören Hilmi çok sevindi." Buradaki "onu" kelimesinin vektörünün cümledeki hangi kelimenin vektörüne yakın olmasını isteriz? "Ali"ye. word2vec'te "onu" kelimesi her cümlede aynı sabit vektörü alır. Oysa "Kedi okula gelince onu gören … çok sevindi" cümlesinde "onu"nun vektörü bu kez "kedi"ye yakın olmalıdır. Bağlam temelli yapılar bir kelimenin bu şekilde iyi temsil edilmesini sağlar.

Self-attention, her kelimenin kendisi dışındaki bütün kelimelerle bağımlılıklarını (i. kelimenin j. kelimeye ne kadar etkisi olacağını gösteren α'ları) hesaplar. Transformer hem girişin hem çıkışın kendi içindeki bağımlılıklarına bakar. Transformer'lar ilk olarak makine çevirisi için önerildi.

7. Bağlam Tabanlı Dil Modelleri

Hoca burada doğal dil işlemedeki önceden eğitilmiş (pre-trained) modeller kısmının başladığını, ama bunun doğrudan BERT ile başlamadığını söyledi. Slayt (ekrandan, 01:30:00) modelleri iki gruba ayırır.

7.1 RNN tabanlı yaklaşımlar

- TagLM: önceden eğitilmiş (sonraki kelimeyi tahmin için eğitilmiş) bir modelin başka bir görevde ilk kullanımı. Hoca bunu önceden eğitilmiş mimarilerin ilk prototipi olarak niteledi: sonraki kelimeyi tahmin etmek için bir mimari eğitilmiş, sonra metin sınıflandırma gibi başka görevlerde kullanılmıştır.
- ELMo: çok sayıda görevde performans artışı. Model sonraki kelimeyi tahmin etmek için eğitilir, mimari korunur ve üzerine küçük bir sınıflandırma katmanı yerleştirilir. Bu katman farklı görevlere göre özelleştirilir. Böylece birçok görev için kullanılabilen ve ince

ayar (fine-tune) yapılabilen tek bir mimari çıkar. Hoca ELMo'yu birçok göreve özelleştirilebilen ilk klasik sinirsel dil modeli olarak tanımladı.

7.2 Transformer tabanlı yaklaşımlar

- GPT (OpenAI): sonraki kelimeyi tahmin.
- BERT: maskelenmiş kelimeyi tahmin ve "sonraki cümle mi" ikili sınıflandırması (pozitif: ardışık iki cümle, negatif: rastgele iki cümle).
- RoBERTa: yalnız maskelenmiş kelime tahmini; BERT'teki sonraki cümle tahmininin pek etkisi olmadığı düşünülüp kaldırılmıştır.
- ALBERT: maskelenmiş kelime tahmini ve sonraki cümle mi ikili sınıflandırması; ama negatif örnek olarak pozitifteki iki cümlenin yeri değiştirilir. A'dan sonra B pozitif, B'den sonra A negatiftir. Hoca bunun daha zor bir soru olduğunu belirtti.
- ELECTRA: değiştirilmiş kelimeleri tahmin. Cümledeki bir kelime rastgele başka bir kelimeyle değiştirilir; model hangi kelimenin değiştirildiğini bulmaya çalışır.
- BART: çeşitli görevler (sonraki bölüm).

7.3 Ön eğitim için etiket gerekmez

Hoca bu görevleri word2vec ile karşılaştırdı ve sordu: word2vec'in eğitimi için etiketleme yapmak gerekir mi? Hayır. Orada bir kelime verilip etrafındaki kelimeler ya da etraftaki kelimeler verilip ortadaki kelime tahmin ediliyordu. GPT'de belli sayıda kelime verilip sonraki kelime tahmin edilir. BERT'te bir cümlenin içindeki bazı kelimeler kapatılır ve sistem bunların ne olduğunu bulmaya çalışır. Derlemden bir cümle seçilip rastgele kelimeler kapatıldığında doğru cevap zaten bilinir; insanların etiketleme yapmasına gerek yoktur ve eğitim verisi üretmek çok basittir.

8. Self-Supervised Görevler

8.1 Tanım ve örnekler

Slayt (ekrandan, 01:40:00):

- Büyük miktarda eğitim verisi üretmek için üretilen yan görevler.
- İnsan etiketlemesine ya da emeğine ihtiyaç yok.
- Sonraki kelime(ler)i, cümleyi ya da film karesini tahmin.
- Gürültü ekleme: maskelenmiş kelime(ler)i tahmin, sırası bozulmuş cümleyi düzeltme, harfleri bozulmuş kelimeyi düzeltme, resmin silinmiş kısmını tahmin.

Hoca film örneğinde bir filmin içindeki belli parçaların kaldırılıp tahmin edilmeye çalışılabileceğini, ama bunun kelimelerle çalışmaktan çok daha zor olduğunu söyledi.

> **Hocanın vurgusu:** Başarılı ve yararlı görevler üretebilirseniz bedavadan bilgi kazanmış olursunuz; bedavadan derken etiketlemeye ihtiyaç duymadan.

8.2 BART görevleri

Slayttaki şemada orijinal cümle "ABC.DE." (üç kelime, nokta, iki kelime, nokta) ve çıkış hep bu orijinal cümledir. Girişler:

- Token Masking: "A_C._E." (bazı kelimeler maske olmuş, ne oldukları tahmin edilir)
- Token Deletion: "A.C.E." (bazı kelimeler silinmiş, nerelerin silindiği bulunur)
- Text Infilling: "A_.D_E." (araları doldurmak)
- Sentence Permutation: "DE.ABC." (cümlelerin yeri karıştırılmış, orijinal sıraya döndürülür)
- Document Rotation: "C.DE.AB" (doküman bir noktadan döndürülmüş)

8.3 Görüntüde self-supervised görevler

Görüntü işlemede resmin bir kısmını silip orayı tahmin etmek bir self-supervised görevdir. Karşıtlıklı öğrenmede (contrastive learning) hoca tek bir resimden eğitim örnekleri üretmeyi

anlattı: büyük resmin içinden rastgele dikdörtgen parçalar seçilir ve “bu iki parça aynı resimden mi?” diye sorulur. Aynı resmin içindeki parçaların birbirine benzer, farklı resimlerdeki parçaların farklı olduğu öğrenilir. Bir başka görev resmi 16 parçaya (4×4) bölüp parçaların yerlerini karıştırmak ve modelden orijinal hâline döndürmesini istemektir.

Bu görevleri yapabilir hâle gelen sistemler alınıp kendi işlerimizde kullanılır; önceden eğitilmiş yapılar böyle üretilir.

9. Byte Pair Encoding

9.1 Algoritma

Byte pair encoding (BPE) aslında veri sıkıştırmak için önerilmiş bir yöntemdir. Slayttaki örnek (ekrandan, 01:42:20):

```
aaabdaaabac
ZabdZabac       (Z=aa)
ZYdZYac         (Y=ab, Z=aa)
XdXac           (X=ZY, Y=ab, Z=aa)
```

Adım adım: ilk dizide ardışık en sık geçen ikili “aa”dır; “aa”lara Z denir. Yeni dizide “ab” sık geçen bir ikilidir; “ab”lere Y denir. Sonra “ZY” ikilisi iki kez geçer; buna X denir. Artık tekrar eden ikili kalmamıştır. Python ile doğrulandı: “ZabdZabac” dizisinde “Za” ve “ab” ikişer kez geçer (eşitlik var, slayt “ab”yi seçmiş); “ZYdZYac” dizisinde “ZY” iki kez geçer ve sonuç “XdXac”tır.

Yöntem, sekansın içinde en sık geçen alt birimleri (subunit) bulur. Bir dil için düşünülürse bu alt birimler kelime parçalarına ya da çok sık kullanılan tekil kelimelere denk gelebilir. Yani frekansı en yüksek kelimeler ve kelime parçaları bulunur.

9.2 Bilinmeyen token sorunu

Sinirsel dil modellerinde BPE kullanmak çok yaygındır; temel sebep bilinmeyen (unknown) token’lardan kurtulmaktır. Kelimeler en küçük birim kabul edilirse eğitim kümesinde olup testte olmayan kelimeler sorun değildir. Asıl sorun eğitimde hiç görülmemiş ama test zamanında karşımıza çıkan kelimelerdir. Bunlar için bir “unknown” token’ı kullanılabilir; bu “burada nadir bir kelime var” demektir ve nadir kelimelerin hepsine aynı kelime vektörü atanır. Hangisinin hangisi olduğu bilinemediği için farklı vektör kullanmanın bir anlamı yoktur.

BPE kullanıldığında bilinmeyen token hiç oluşmaz, çünkü en küçük birim karakterdir. Hocanın slayttaki örneği (ekrandan, 01:45:50): “İlkbaharxdefsr” gibi uydurma bir kelimede “İlk” ve “bahar” sözlükte bulunan alt token’lardır; geri kalan yaygın olmayan kısım karakterlere bölünür: İlk + bahar + x + d + e + f + s + r. Her birinin bir vektörü olduğu için ne yazılırsa yazılsın bilinmeyen token oluşmaz. Hiç tanımlanmamış bir karakter (farklı bir kodlama) kullanılırsa bu ön işlemede çözülecek ayrı bir sorundur.

9.3 Kaynaklar ve Türkçe için önemi

Slayt (ekrandan, 01:50:00): BPE dil işleme için “Neural Machine Translation of Rare Words with Subword Units” makalesiyle popüler oldu. WordPiece ve SentencePiece (ücretsiz ve hızlı) gibi kütüphaneler vardır.

Slayttaki soru: “Türkçe için sadece OOV değil, normal kelimeler için de çok önemli. Neden?” Örnekler: gel-ecekmiş, bil-ecekmiş, bit-miş, gül-müştü. Türkçe gibi eklemeli dillerin en büyük sorunu, aynı kelimenin sonuna gelen farklı eklerle anlamın değişmesi ve kelime sayısının korkunç derecede artmasıdır. Daha önce bunun için Zemberek gibi morfolojik analiz araçları kullanılıyor, kelime gövde ve eklerine ayrılıyordu.

BPE’de sözlüğün ne olacağı bir derlem verilerek ürettirilir (“şu derlemi al, bana 64 binlik bir sözlük üret”), tokenizer da kelimeleri bu sözlüğe göre parçalar. Türkçe için yapıldığında parçalar morfolojik analizden çıkmış gibi görünür, ama değildir: “ecekmiş”, “miş”, “müştü” gibi ifadeler çok sık kullanılan alt birimler olduğu için ortaya çıkarlar. Sistem Türkçe bilmez,

tamamen frekans tabanlıdır. Sözlükte “gelecekmiş” olmasa da “gel” ve “ecek”, “miş” varsa sayısız kombinasyonun hepsi temsil edilebilir.

> **Hocanın vurgusu:** BPE Türkçe için çok daha önemli hâle geldi ve bizi bir şekilde morfolojik analiz yapmaktan kurtardı. Morfolojik analiz başka amaçlar için yine gerekebilir.

10. Büyük Resim ve Hazır Modeller

Hoca “Büyük Resim” slaytında modellerin soy ağacını gösterdi: ELMo, ULMFiT, GPT, BERT ve ardından GPT-2, GPT-3. Parametre sayısında bir patlama yaşandığını söyledi.

> **Ders sonrası not:** Hoca GPT-3’ün parametre sayısı için “trilyona yaklaştı herhalde” dedi. GPT-3 büyük olasılıkla 175 milyar parametrelidir.

Görsel yapılarda da gelişme vardır: Transformer’ların içinde RNN olmadığı gibi CNN de yoktur ve CNN ile çözülen görüntü işleme problemlerini Transformer’larla çözme üzerine epey yol alınmıştır. Hocanın ifadesiyle Transformer’lar hem RNN’lerin hem CNN’lerin “pabucunu dama atmaya” uğraşıyor.

Hazır modeller slaytında Türkçe için örnekler vardır: metin sınıflandırma, eksik kelime tahmini, Türkçe için BERT’i kullanma ve eğitme, GPT-2’nin Türkçesi, Türkçe derlemlerle eğitilmiş BERTurk. Bunlar alınıp kendi modellerle hazır kullanılabilir.

11. BERT Nasıl Çalışır?

Aradan sonra hoca, bir YouTube videosunu izlerken aldığı notları (“nasıl_calisir_BERT” belgesi) paylaştı.

11.1 ELMo, GPT ve BERT

Belgedeki şekil BERT makalesinden alınmıştır (ekrandan, 02:10:00). ELMo bağımsız eğitilmiş soldan sağa ve sağdan sola LSTM’leri birleştirir; LSTM de bir RNN olduğu için ELMo RNN mimarisidir. OpenAI GPT soldan sağa bir Transformer kullanır; her kelime yalnız kendinden önceki kelimelerle ilişkilidir, çünkü her şey sonraki kelimeyi tahmin etmeye yöneliktir. BERT’te her kelime her kelimeyle ilişkilidir; bu yüzden makalede “bidirectional Transformer” denir. Hoca GPT’yi bu açıdan BERT’in daha az şey öğrenen özel bir hâli gibi düşünebileceğimizi söyledi.

11.2 Self-attention: key, value, query

Hocanın notu: “her kelime için Key ve Value vektörleri üretiliyor. Query ise hedef kelimenin vektörü. q ile k’nın inner product’ı ile benzerlikleri bulunuyor. Mesela aşağıda q’ya en benzeyen kelime b. Bu benzerliklerle kelimelerin v’leri ağırlıklandırılıyor.”

Girişte her token için ham bir gömme ($E_1, E_2, \ldots$) vardır; Transformer bunları katman katman işleyip en sonda çok daha iyi temsiller üretir. Bu temsiller üretilirken self-attention kullanılır. Her kelime için üç vektör vardır: key, value ve query. Bunlar kelimenin mevcut temsilini öğrenilen W_K, W_V, W_Q matrisleriyle çarparak elde edilir; matrisle çarpmak bir izdüşüm (projeksiyon) yapmaktır. Hoca orijinal yapıda 512 boyutlu temsilin bu vektörler için 64 boyuta indirildiğini, sonra çok başlı (multi-head) yapıyla tekrar 512’ye çıkarıldığını söyledi.

Adımlar:

- a, b, c, d, e kelimeleri için key ve value vektörleri üretilir.
- Hedef kelimenin (ör. “onu”) query vektörü bütün key’lerle iç çarpıma (dot product) sokulur. Her çarpım tek bir sayı verir: query’nin o key’e ne kadar benzediği. Bu değerler softmax’tan da geçirilir.
- Hedef kelimenin yeni temsili, value vektörlerinin bu benzerliklerle ağırlıklandırılmış toplamıdır. Hocanın çizimindeki örnekte q, b’ye çok, a, c, d’ye biraz, e’ye en az benzer; yeni temsil en çok b’nin value’sundan gelir.

Bu yapı aynı kelimenin her cümlede farklı değerler alabilmesini sağlar. Hoca neden üçlü bir yapı kullanıldığına dair matematiksel bir zorunluluk olmadığını, bunun uygulamada iyi sonuç veren bir tasarım olduğunu söyledi.

11.3 Giriş gömmeleri ve konum bilgisi

Hocanın notu (ekrandan, 02:20:00): giriş gömmeleri token embedding (word2vec benzeri), segment embedding (2 cümleyi ifade ediyor) ve position embedding'dir (kelimelerin yerlerini kodluyor). Şekilde giriş "[CLS] my dog is cute [SEP] he likes play ##ing [SEP]" dizisidir. "##ing", tokenizer'dan gelen bir alt kelime parçasıdır; OOV token'larını ortadan kaldırmak için kelimeler değil alt kelimeler token olarak alınır.

- Token embedding: BPE ile bulunan parçaların her birinin gömmesi.
- Segment embedding: birden fazla cümleden oluşan girişlerde birinci cümleyi (A) ve ikinci cümleyi (B) ayırt etmeye yarar; ayırıcıyla (separator) birlikte kullanılır.
- Position embedding: asıl ilginç olan budur. RNN'lerde kelimeler sırayla verildiği için konum bilgisine gerek yoktur. Transformer'da her şey tek seferde verilir ve tekrarlayan bir yapı yoktur; sistem bir kelimenin cümlenin başında mı ortasında mı olduğunu bilemez, her şey bir bag of words gibi görünür. Konum gömmeleri bundan kurtarır.

Hoca konum gömmesini tahtada farklı frekanslı sinüslerle çizdi: yatay eksen konum, her satır bir sinüs fonksiyonu, her sütun bir kelime. Gerçekte örneğin 50 sinüs olur ve her kelime bu 50 fonksiyonun o konumdaki değerleriyle temsil edilir. Yavaş değişen sinüste birbirine yakın kelimelerin (aralarında bir, iki, üç kelime olanların) değerleri yakın, çok uzaktakilerin uzaktır; bu, kelimenin kabaca cümlenin başında mı sonunda mı olduğunu ifade eder. Daha hızlı sinüs daha dar bir alanda birliktelikleri ifade eder. En hızlı sinüste yan yana iki kelimenin bile değerleri farklıdır; hangisinin önce geldiği ayırt edilir. Yani konum bilgisi farklı çözünürlüklerde kodlanır.

> **Ek bilgi (derste anlatılmadı):** Sinüs ve kosinüs tabanlı konum kodlaması "Attention is all you need" makalesindeki Transformer'da kullanılır. BERT'te ise konum gömmeleri büyük olasılıkla eğitim sırasında öğrenilen vektörlerdir; sezgi (her konuma ayırt edilebilir bir vektör vermek) aynıdır.

11.4 Ön eğitim: NSP ve MLM

BERT'in bütün ön eğitim dizileri [CLS] token'ıyla başlar; cümleler [SEP] ile ayrılır. Next Sentence Prediction'da iki cümlelik bir giriş için ikinci cümle birinciden sonra geliyorsa etiket "IsNext", rastgele seçilmiş başka bir cümleyse "NotNext"tir. Aynı girişte bazı token'lar maskelenmiştir ve onların ne olduğu da tahmin edilir.

Hocanın notu (ekrandan, 02:36:40), "kaynak videoda anlatılmayan" kısımlar:

- NSP zaten bir cümle çifti sınıflandırma görevidir: [CLS] token'ının çıkış vektörü C kullanılarak sınıflandırma yapılır, bunun için bir W öğrenilir.
- MLM için makaleden alıntı: "In this case, the final hidden vectors corresponding to the mask tokens are fed into an output softmax over the vocabulary, as in a standard LM. In all of our experiments, we mask 15% of all WordPiece tokens in each sequence at random."
- Yani giriş cümlesindeki bazı token'lar maskeye çevrilip BERT'e verilir. BERT her token için bir vektör üretir. Maskelilere karşılık gelen vektörler için bir W öğrenilir, ama bu W büyüktür: $W \in \mathbb{R}^{V \times H}$, sınıf sayısı V sözlük büyüklüğüdür. Maskeli konum için üretilen vektör yaklaşık 30 bin sınıftan birine atanır. İşlem: $C \in \mathbb{R}^{1 \times H}$, $W \in \mathbb{R}^{V \times H}$ olmak üzere $\text{softmax}(CW^T) \in \mathbb{R}^{1 \times V}$.
- Özetle: iki cümleden oluşan, birbirini takip eden ya da etmeyen bir dizi oluşturulur; bazı kelimeler maskelenir ve tek dizi olarak ağa verilir. Çıkışta [CLS]'nin vektörü NSP için bir W ile, maskeli kelimelerin vektörleri başka bir W ile MLM için çarpılır. Eğitim kaybı bu tahminlerin kayıplarının toplamıdır.

Ön eğitimde mimarinin üstünde bu ince sınıflandırma katmanı oluşturulur ve BERT'in kendisiyle birlikte eğitilir. Ön eğitim bitince bu katman kaldırılır. İnce ayarda, BERT'in nerede kullanılacağına göre üstüne göreve özel küçük bir sınıflandırma katmanı eklenir.

12. BERT'in İnce Ayarı ve Kullanımı

12.1 Cümle çifti sınıflandırma

Görev: sisteme iki cümle verilir, aralarındaki ilişkinin tipi bulunur. Örnekler: bu iki cümle eş anlamlıdır / değildir; alakasızdır / alakalıdır; aynı kişi tarafından yazılmıştır / farklı kişiler tarafından yazılmıştır.

Giriş [CLS], birinci cümlenin token'ları, [SEP], ikinci cümlenin token'larından oluşur. BERT her token'ın giriş gömmesini (token, segment ve konum gömmelerinin toplamı) oluşturur ve Transformer'a verir. Arada 12 katmanlı (12 tane üst üste Transformer) bir yapı vardır; temsiller katman katman dönüşür. Son katmanda [CLS]'ye karşılık gelen çıkış C'dir ve sınıf etiketi için kullanılır.

Yapılacak iş: C vektörünü (hocanın örneğinde 512 boyutlu) bir W ile çarpmak. 3 sınıf varsa W 512×3, 10 sınıf varsa 512×10, 2 sınıf varsa 512×2 boyutludur. İnce ayar temelde bu W'yi öğrenmekle sınırlıdır. Tüm ağın ağırlıklarını da bu süreçte değiştirmek mümkündür; bu tam bir ince ayar olur ve C'nin daha iyi üretilmesi için önceki katmanlar da güncellenir.

> **Hocanın vurgusu:** Cümle çifti sınıflandırmada öğrenilen şey, BERT'in kendi içindeki 300 küsur milyon parametreye ek olarak yalnız bir matristir; bu da devede kulak kalır.
>
> **Ders sonrası not:** Hoca örneklerde C'yi 512 boyutlu aldı. BERT-Base'in gizli boyutu büyük olasılıkla 768, BERT-Large'ınki 1024'tür; 512, BERT'in kabul ettiği en uzun giriş dizisinin token sayısıdır. Hesabın mantığı (W = gizli boyut × sınıf sayısı) değişmez.

12.2 Soru cevaplama

Hocanın notu (ekrandan, 02:30:00): "sorunun cevabı paragrafın neresinde başlayıp bitiyor bulacağız. Paragrafın her token'ı için bir sonuç token'ı üretiliyor: $T'_1, T'_2, \ldots, T'_M$. Bunların her biri mesela 512 boyutlu vektörler. 2 query var: Q_{start}, Q_{end}. Bunların her birinin $T'_1, T'_2, \ldots, T'_M$'lerle inner product'ları alınıp start olma ve end olma olasılık dağılımları bulunuyor. Buradaki finetune işlemi Q_{start} ve Q_{end} vektörlerini öğrenmek."

Veri kümesi SQuAD v1.1'dir. Girişin ilk kısmında soru, [SEP]'ten sonraki ikinci kısmında paragraf verilir (BERT iki cümle alabildiği için). Öğrenilen iki vektör, sınıflandırmadaki W matrisi yerine geçer:

- Q_{start}, paragraf token'larının bütün çıkış vektörleriyle iç çarpılır; en yüksek değeri veren token cevabın başlangıcıdır.
- Q_{end} ile aynı işlem yapılır; en yüksek değeri veren token cevabın sonudur.

Bu iki vektör her soru için aynıdır; değişen, her girişte paragraf token'larının vektörleridir. Hoca makalenin güzel ve okuması keyifli olduğunu, ayrıntıların orada bulunduğunu söyledi.

12.3 Adlandırılmış varlık tanıma (NER)

Görevde tek bir cümle girer (CoNLL-2003 NER); iki cümlelik yapı yoktur. Cümlenin her token'ı için üretilen çıkış vektörleri ($T_1, T_2, \ldots, T_N$) NER etiketlerine atanır; bunun için bütün token'larda ortak kullanılan tek bir W öğrenilir. Cümle çifti sınıflandırmaya çok benzer; fark, orada yalnız C için, burada her token için tahmin yapılması ve sınıf sayısının varlık türü sayısı kadar olmasıdır. Hoca bunun en fazla kırk kadar olduğunu, üç beş türlü olanların da bulunduğunu söyledi.

12.4 Çoktan seçmeli sorular

Hocanın notu (ekrandan, 02:40:00): "SWAG dataseti 'bu cümle aşağıdaki şıklardan hangisi ile devam eder' şeklinde oluşturulmuş. Çoktan seçmeli bu gibi datasetler için finetuning'de 4 (cevap şıkkı sayısı) girdi oluşturuluyor. Bu girdilerin her biri 2 kısımdan oluşuyor. İlk kısımlar aynı ve soru cümlesi, 2. kısımlar ise cevap şıkları. Burada CLS'nin çıkış vektörü olan C ile dot product yapılacak bir vektör öğreniliyor. 4 sekansın çıkışlarına bakılarak hangisi büyükse o cevap olarak dönülüyor."

Hocanın tahtadaki gösterimi:

```
soru + a şıkkı  ->  Ca
soru + b şıkkı  ->  Cb
soru + c şıkkı  ->  Cc
soru + d şıkkı  ->  Cd
öğrenilen tek vektör: R
skorlar: R·Ca, R·Cb, R·Cc, R·Cd  ->  en büyüğü cevap
```

Örneğin $R \cdot C_c$ en büyükse sistem cevabın c şıkkı olduğunu bulur.

12.5 Feature-based kullanım

BERT'in ince ayarında tüm ağ (large'da yaklaşık 300 milyon, base'de 110 milyon parametre) güncellenir ve her ince ayar sonunda o göreve özel bir ağ çıkar. Ama bu şart değildir. Hocanın notu (ekrandan, 02:43:15): "Bir sekansın BERT temsilini alıp bu temsilleri başka mimariler içinde kullanmayı feature-based yaklaşım olarak adlandırmışlar. Bu da denemelerde hiç fena çıkmamış."

Yani token'lar (tek cümle, birden fazla cümle ya da paragraf) verilir, çıkış temsilleri alınır ve istenen makine öğrenmesi modeline ya da doğal dil işleme kütüphanesine verilir. Üstteki sınıflandırma kısmı BERT'ten koparılır, ayrı bir yerde kendi başına eğitilir.

Bir ayrıntı: 12 katmanlı mimaride her katman her token için bir çıkış vektörü üretir; yani bir giriş dizisi için 12 adet temsil üretilir. En son katmandaki temsili almak, sondan bir öncekini almak, son 4 katmanı birleştirmek (concatenate) gibi yaklaşımların performansı NER veri kümesi üzerinde denenmiştir (makalede tablo 7). Sondan bir önceki katman ve son 4 katmanı birlikte kullanmak, son katmandan daha iyi sonuç vermiştir.

Neden son katman daha iyi değil? Makaleye göre son katman ön eğitim sırasında göreve özgü olarak üretilir; masked word prediction ve next sentence prediction görevlerine fazla odaklanmıştır. Bir önceki katman daha genel geçer problemler için daha iyidir.

Hocanın notunda bir ayrıntı daha yer alır: her token için her katmanda H boyutlu bir vektör üretilir; farklı uzunluktaki giriş dizilerinin temsilleri farklı uzunlukta olur. Aynı hâle getirmek için ortalama alınır: tüm token'ların 1. boyutlarının ortalaması dizinin temsilinin 1. boyutu, 2. boyutlarının ortalaması 2. boyutu olur.

12.6 Özet: BERT bir encoder'dır

Hocanın özeti (ekrandan, 02:43:15): BERT bir encoder'dır; bir sekans verilir, o sekans için bir temsil üretir. "Attention is all you need" makalesinde Transformer'lar makine çevirisi özelinde anlatılmıştı ve orada giriş dilindeki cümleyi temsil etmek için bir encoder, çıkış dilindeki cümleyi üretmek için bir decoder vardı (RNN'lerdeki encoder-decoder gibi). BERT bir dilden başka dile çeviri yapmak için tasarlanmadı; sekanstan sekans üretmez, temsil üretir ve bu temsille bambaşka problemler çözülebilir. Yalnız encoder kısmı vardır ve içindeki self-attention yapısı o makaledeki encoder ile aynıdır.

Ön eğitimde bu mimarinin üzerine basit bir sınıflandırma katmanı eklenir, eğitim bitince çıkarılır. İnce ayar süreçlerinde ise mimarinin üzerine göreve özgü yine basit bir sınıflandırma katmanı eklenir.

Örnek Sorular

Soru 1 (ekrandan, 00:07:40): İnsanlar kadın/erkek, gözlüklü/gözlüksüz, uzun/kısa diye sınıflandırılacak. Kaç sınıf tanımlanır? Alt özelliklerle çalışan yaklaşımın avantajı nedir? 1024 sınıf için kaç alt özellik gerekir?

> **Çözüm (derste verildi):** Üç ikili özellik $2^3 = 8$ sınıf verir ve her sınıftan örnek gerekir. Üç ikili sınıflandırıcıyla eğitim kümesinde "uzun, kadın, gözlüksüz" örneği olmasa bile bu kombinasyon tanınır. d alt özellik 2^d kombinasyon ifade ettiği için N örtüşmeli sınıf $\log_2 N$ alt özellikle ifade edilir; $N = 1024$ için $\log_2 1024 = 10$ (python ile doğrulandı). Slayttaki ikinci örnekte de 10 alt özellik kullanılır.

Soru 2 (ekrandan, 00:10:00): 1024 sınıf ve $d \times d$'lik resimler için, $d = 128$ iken klasik yaklaşım ile 10 alt özellikli ara katmanlı yaklaşımın öğrenmesi gereken parametre sayılarını karşılaştırın.

> **Çözüm (derste verildi, sayı düzeltildi):** Klasik: $(d \cdot d) \cdot 1024 = 16\,384 \cdot 1024 = 16\,777\,216 = 2^{24}$. Dağıtık: $(d \cdot d) \cdot 10 + 10 \cdot 1024 = 163\,840 + 10\,240 = 174\,080 \approx 180$ bin. Slaytta klasik değer "yaklaşık 4 milyon" diye yazılmış; doğrusu yaklaşık 16,8 milyondur (python ile hesaplandı). Ara katman parametre sayısını yaklaşık 96 kat azaltır.

Soru 3 (derste sözlü, 00:15:36): 1024 sınıfın hiçbir ortak parçası yoksa (ör. 1024 ilçenin resimleri) dağıtık temsil yine işe yarar mı?

> **Çözüm (derste verildi):** Dağıtık temsil, bütünlerin temel bileşenlerden oluşmasına dayanır. Temel bileşenler yoksa yapılacak bir şey yoktur, sistem bir arama tablosuna dönüşür. Renklerde olduğu gibi (1024 renk, birkaç temel rengin farklı oranları) temel bileşenler varsa yöntem çalışır.

Soru 4 (derste sözlü, 00:35:42): Kelime vektörleri yan yana dizilip CNN uygulanınca, farklı kelimelerin aynı indeksli elemanları yan yana gelir. Bu anlamsız bir ilişki kurmaz mı?

> **Çözüm (derste verildi):** Küçük (2×2, 3×3) çekirdekler kaydırılsaydı kurardı. Ama çekirdek kelime boyutunun tamamını kapsayacak biçimde (300×3) seçilir ve yalnız kelime ekseninde kaydırılır. Satırlar bütün kelimelerin aynı boyutlarıdır, yan yanalık zaten var olan kelime sırasıdır; olmayan bir ilişki eklenmez.

Soru 5 (ekrandan, 00:44:40): Bir sonraki harf yerine bir sonraki kelimeyi tahmin etmek için one-hot kullansak ne olur? Alternatif ne olabilir?

> **Çözüm (derste verilmedi):** Büyük olasılıkla beklenen yön şudur: kelime düzeyinde one-hot vektörün boyutu sözlük büyüklüğü kadar (on binlerce) olur, çok seyrek olur ve kelimeler arası benzerliği taşımaz. Alternatif olarak girişte derste daha önce görülen yoğun kelime vektörleri (word2vec gibi) kullanılabilir; sözlük sorununu azaltmak için de bu derste anlatılan alt kelime (BPE) birimleri kullanılabilir.

Soru 6 (derste sözlü, 00:51:58): Encoder-decoder'da ilk çıkış kelimesi üretildikten sonra ikinci kelime nasıl üretilir?

> **Çözüm (derste verildi):** Temel yapıda ilk kelime o anki durumdan W^S ile üretilir, sonra durum W^{hh} ile güncellenir ve yeni durumdan ikinci kelime üretilir. Üretilen kelimenin kendisi bir sonraki adıma verilmez. İyileştirilmiş yapıda ise $h_t = \varphi(h_{t-1}, C, y_{t-1})$ ile önceki çıktı ve bağlam vektörü de kullanılır.

Soru 7 (derste sözlü, 01:22:03): 20 kelimelik bir giriş ve 20 kelimelik bir çıkış için klasik attention'da kaç katsayı öğrenilir?

> **Çözüm (derste verildi):** Her (çıkış, giriş) çifti için bir $\alpha^{i,j}$ vardır: $20 \cdot 20 = 400$ katsayı. Genel olarak giriş token sayısı çarpı çıkış token sayısı.

Soru 8 (derste sözlü, 01:27:00): "Ali okula gelince onu gören Hilmi çok sevindi" cümlesinde "onu" kelimesinin vektörü hangi kelimenin vektörüne yakın olmalıdır? word2vec bunu sağlar mı?

> **Çözüm (derste verildi):** "Ali"ye. word2vec "onu"ya her cümlede aynı sabit vektörü verir; "Kedi okula gelince …" cümlesinde de aynıdır. Self-attention ile bağlam farkında modeller "onu"nun vektörünü cümleye göre ("Ali" ya da "kedi") değiştirir.

Soru 9 (derste sözlü, 01:32:44): word2vec'in eğitimi için etiketleme gerekir mi? BERT'in maskelenmiş kelime görevi için?

> **Çözüm (derste verildi):** İkisinde de gerekmez. word2vec bir kelimeden etrafındakileri ya da etraftakilerden ortadakini tahmin eder. BERT'te derlemden seçilen cümlede rastgele kelimeler kapatılır; doğru cevap zaten bilinir.

Soru 10 (ekrandan, 01:42:20): "aaabdaaabac" dizisine byte pair encoding uygulayın.

Çözüm (derste verildi): En sık ikili “aa” → Z: “ZabdZabac”. Sonra “ab” → Y: “ZYdZYac”. Sonra “ZY” → X: “XdXac”. Tekrar eden ikili kalmaz. Python ile doğrulandı; ikinci adımda “Za” ile “ab” eşit sıklıktadır (ikişer), slayt “ab”yi seçmiştir.

Soru 11 (ekrandan, 01:50:00): Byte pair encoding Türkçe için neden yalnız OOV kelimeler için değil, normal kelimeler için de çok önemlidir?

Çözüm (derste verildi): Türkçe eklemeli bir dildir; aynı gövdeye gelen farklı eklerle kelime sayısı korkunç derecede artar. BPE derlemden frekansa dayalı alt birimler çıkarır: gel-ecekmiş, bil-ecekmiş, bit-miş, gül-müştü. “ecekmiş”, “miş”, “müştü” gibi ekler sık geçtiği için ayrı token olur; böylece sözlükte olmayan çekimli kelimeler de temsil edilir ve morfolojik analiz gereği büyük ölçüde ortadan kalkar.

Soru 12 (ekrandan, 02:36:40): BERT’in MLM ön eğitiminde maskeli bir konumun çıkış vektörü $C \in \mathbb{R}^{1\times H}$ ise W’nin boyutu ve çıkışın boyutu nedir?

Çözüm (derste verildi): $W \in \mathbb{R}^{V\times H}$ (V: sözlük büyüklüğü, yaklaşık 30 bin). Çıkış softmax$(CW^T) \in \mathbb{R}^{1\times V}$; maskeli konum sözlükteki V token’dan birine atanır. Sınıflandırmadaki $H \times 3$ gibi küçük bir W’den tek farkı sınıf sayısının sözlük büyüklüğü olmasıdır.

Soru 13 (ekrandan, 02:40:00): Dört şıklı bir çoktan seçmeli soru (SWAG) BERT ile nasıl çözülür?

Çözüm (derste verildi): Dört girdi oluşturulur: “soru + a şıkkı”, “soru + b şıkkı”, “soru + c şıkkı”, “soru + d şıkkı”. Her biri için [CLS] çıkışı C_a, C_b, C_c, C_d alınır. Öğrenilen tek bir R vektörüyle iç çarpımlar hesaplanır; en büyük $R \cdot C$ değerini veren şık cevaptır.

Dersin Toparlaması

Ders, kelimeye bağlamından bağımsız tek bir vektör veren anlamsal uzayların çok anlamlılık sorunuyla başladı ve bağlama göre değişen temsillere giden yolu adım adım izledi. Önce derin ve katmanlı yapıların neden işe yaradığı dağıtık temsille açıklandı: ortak alt özellikler öğrenilirse sınıflar bunların kombinasyonuyla ifade edilir ve parametre sayısı azalır. Ardından sıralı veriler için RNN’ler, onların çevirideki encoder-decoder kullanımı ve bütün cümleyi tek vektöre sıkıştırma darboğazı görüldü. LSTM ve GRU’nun kapı mekanizmaları, attention’ın her çıkış için girişlere farklı ağırlık vermesi ve RNN’i tamamen bırakan Transformer’lar bu darboğazı aşma adımlarıydı.

İkinci bölümde etiket gerektirmeyen self-supervised ön eğitim görevleri (sonraki kelime, maskelenmiş kelime, sonraki cümle, BART’ın gürültü giderme görevleri, görüntüde karşıtlıklı öğrenme) ve bilinmeyen kelime sorununu çözen, Türkçe için ayrıca önemli olan byte pair encoding anlatıldı. Son bölümde BERT’in self-attention (query, key, value) yapısı, giriş gömmeleri, MLM ve NSP ile ön eğitimi, cümle çifti sınıflandırma, soru cevaplama, NER ve çoktan seçmeli sorular için ince ayarı ve feature-based kullanımı ele alındı. Özetle BERT, metne bağlam farkında temsil üreten bir encoder’dır ve üzerine eklenen küçük göreve özgü katmanlarla birçok problemde kullanılır.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.